Lesson 3: OpenGL Programming

Lesson 3: OpenGL Programming 1
3.1 Introduction 2
Capabilities sand Features of OpenGL 2
3.2 The GLUT 3
3.2.1 OpenGL Utility Toolkit 3
3.3 An Illustrative Example 3
3.4 Modelling and Matrices in OpenGL 5
3.5 A Simple Animation – Spinning 9
Lesson 3 Discussion Session 9

3.1 Introduction

OpenGL

OpenGL is a worldwide standard for computer graphics programming. It is a set of routines, protocols, and tools for building computer graphics.

It's the **specification** of an Application Programming Interface (API) for computer graphics programming. OpenGL gained rapid acceptance as a **standard** for computer graphics programming due to two major characteristics. An API specifies how software components should interact and APIs are used when programming graphical user interface (GUI) components 3D computer graphics programming standards.

It's platform-independent, that is it's can work across a whole range of computer hardware.

Portable

To create your graphics, you need an **implementation** of OpenGL.

HISTORY AND VERSIONS OF OpenGL

Version and year	Major functionality changes
OpenGL 1.0, 1992	The first version, with fixed functionality rendering pipeline.
OpenGL 1.1, 1997	Extensions including improved pixel blending, texture effects.
OpenGL 2.0, 2004	Introduction of the programmable rendering pipeline, using GLSL, the OpenGL shading language.
OpenGL 3.0, 2008	Fixed function pipeline deprecated.
OpenGL 4.0, 2010	Changes to functionality to allow OpenGL programs access to hardware features designed for Direct3D.

OpenGL Versions

a) OpenGL ES

OpenGL for Embedded Systems (OpenGL ES) is a subset of OpenGL, used in embedded systems, such as mobile phones and devices. OpenGL ES 1.0 was released in 2003. Examples of implementations include Android, iOS, Blackberry and Raspberry Pi. OpenGL ES 1.x uses fixed functionality rendering; vesions 2.x and above use programmable rendering.

b) WebGL

WebGL is a version of OpenGL ES 2.0 for the Web, using a JavaScript API and the HTML5 canvas element. Development of WebGL started in 2009, and WebGL 1.0 was released in March 2011. Rendering is implemented entirely in the browser.

Capabilities sand Features of OpenGL

a. It has 3D geometric objects, such as lines, polygons, triangle meshes, spheres, cubes, quadric surfaces, NURBS curves and surfaces.
b. It provides 3D modelling transformations, and viewing functions to create views of 3Dscenes using the idea of a **virtual camera**.

c. It supports high-quality rendering of scenes, including hidden-surface removal, multiple light sources, material types, transparency, textures, blending, fog.
d. It provides display lists for creating graphics caches and hierarchical models. It also supports the interactive 'picking' of objects.
e. It supports the manipulation of images as pixels, enabling frame-buffer effects such as anti-aliasing, motion blur, depth of field and soft shadows.

How OpenGL Works

1. Specify geometric objects

2. Describe object properties

• Color

• How objects reflect light

3. Define how objects should be viewed

• where is the camera

• what type of camera

4. Specify light sources

• where, what kind

5. Move camera or objects around for animation

3.2 The GLUT

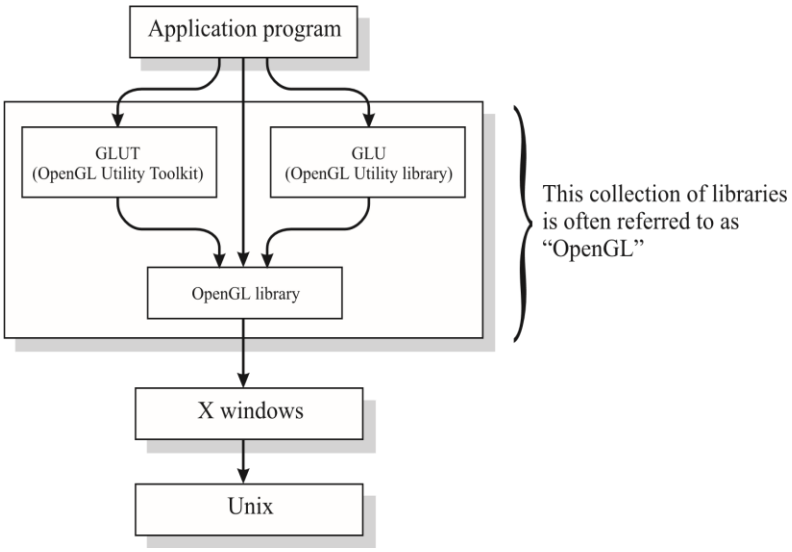
3.2.1 OpenGL Utility Toolkit

This is a utility library called **OpenGL Utility Toolkit**, it was developed to extend the low-level (but very efficient) functionality of OpenGL.

- ✓ Used for managing windows on the display screen.
- ✓ Provides functions for handling input events from the mouse and keyboard.
- ✓ Provides the facilities for interaction that OpenGL lacks.
- ✓ It provides some rudimentary tools for creating Graphical User Interfaces (GUIs).
- ✓ It also includes functions for conveniently drawing 3D objects like the platonic solids, and a teapot.

NB All GLUT function names start with **'glut'**

3.3 An Illustrative Example

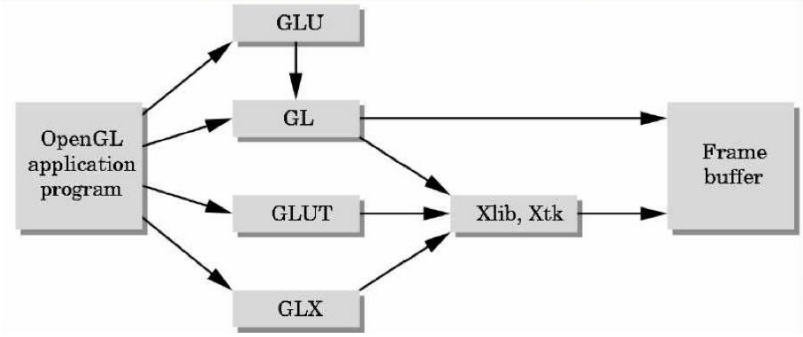


The OpenGL is at a high-level view.

OpenGL Library Organization

- ✓ GL (Graphics Library): core graphics capabilities
- ✓ GLU (OpenGL Utility Library): utilities on top of GL
- ✓ GLUT (OpenGL Utility Toolkit): input and windowing(output)

Recall that OpenGl is the Intermediary between applications and graphics hardware



Application areas of OpenGL

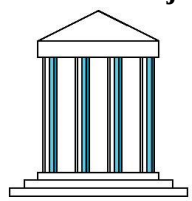
- ✓ CAD- Computer Aided Design
- ✓ Virtual reality
- ✓ Scientific visualization
- ✓ Flight simulation
- ✓ Video Games

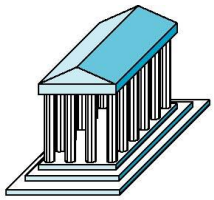
3.4 Modelling and Matrices in OpenGL

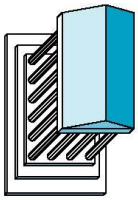
Classical Viewing

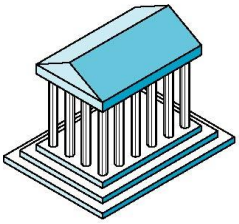
- Viewing requires three basic elements
  - One or more objects
  - A viewer with a projection surface
  - Projectors that go from the object(s) to the projection surface
- Classical views are based on the relationship among these elements
  - The viewer picks up the object and orients it how she would like to see it
- Each object is assumed to constructed from flat *principal faces*
  - Buildings, polyhedra, manufactured objects

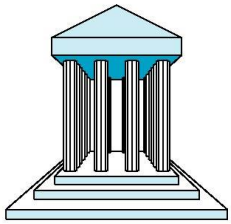
Classical Projections

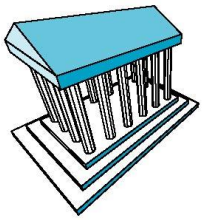

Front elevation


Elevation oblique


Plan oblique


Isometric

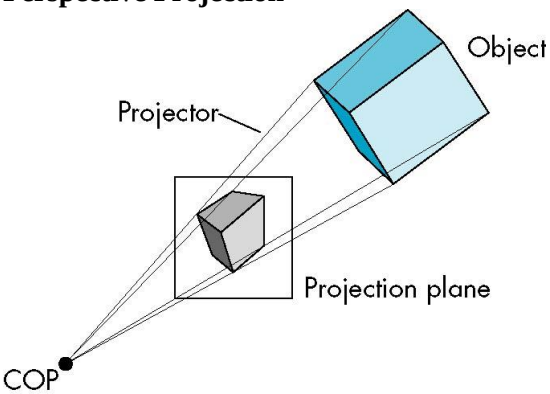

One-point perspective


Three-point perspective

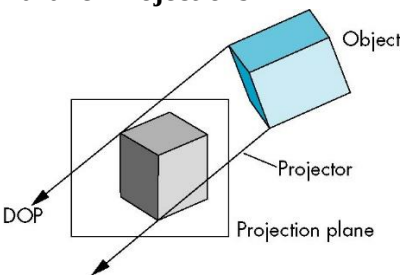
Perspective vs Parallel

- Computer graphics treats all projections the same and implements them with a single pipeline
- Classical viewing developed different techniques for drawing each type of projection
- Fundamental distinction is between parallel and perspective viewing even though mathematically parallel viewing is the limit of perspective viewing

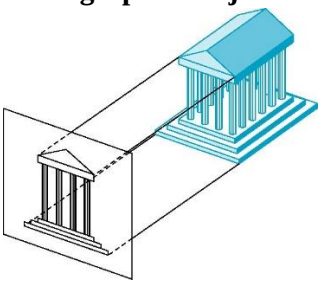
Perspective Projection



Parallel Projections



Orthographic Projection

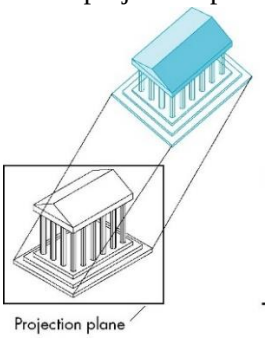


Advantages and Disadvantages

- Preserves both distances and angles
  - Shapes preserved
  - Can be used for measurements
    - Building plans
    - Manuals
- Cannot see what object really looks like because many surfaces hidden from view
  - Often we add the isometric

Axonometric Projections

Allow projection plane to move relative to object



Advantages and Disadvantages Axonometric Projections

- Lines are scaled (*foreshortened*) but can find scaling factors
- Lines preserved but angles are not
  - Projection of a circle in a plane not parallel to the projection plane is an ellipse
- Can see three principal faces of a box-like object
- Some optical illusions possible
  - Parallel lines appear to diverge
- Does not look real because far objects are scaled the same as near objects
- Used in CAD applications

Homogenous Coordinates.

- To convert a 2×2 matrix to 3×3 matrix, we have to add an extra dummy coordinate W.In this way, we can represent the point by 3 numbers instead of 2 numbers, which is called Homogenous Coordinate system. In this system, we can represent all the transformation equations in matrix multiplication.

- Add an extra coordinate, W, to a point.
  - *P(x,y,W).*
- Two sets of homogeneous coordinates represent the same point if they are a multiple of each other.
  - (2,5,3) and (4,10,6) represent the same point.
- If W≠ 0 , divide by it to get Cartesian coordinates of point (x/W,y/W,1).
- If W=0, point is said to be at infinity.

Product matrix as used in geometric transformation

When we perform 2 translations on the same point

$$P' = T(d_{x1}, d_{y1}) \cdot P$$
$$P'' = T(d_{x2}, d_{y2}) \cdot P'$$
$$P'' = T(d_{x1}, d_{y1}) \cdot T(d_{x2}, d_{y2}) \cdot P = T(d_{x1} + d_{x2}, d_{y1} + d_{y2}) \cdot P$$

So we expect :

$$T(d_{x1}, d_{y1}) \cdot T(d_{x2}, d_{y2}) = T(d_{x1} + d_{x2}, d_{y1} + d_{y2})$$

The matrix product $T(d_{x1}, d_{y1}) \cdot T(d_{x2}, d_{y2})$ is :

$$\begin{bmatrix} 1 & 0 & d_{x1} \\ 0 & 1 & d_{y1} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & d_{x2} \\ 0 & 1 & d_{y2} \\ 0 & 0 & 1 \end{bmatrix} = ?$$

Matrix product is variously referred to as *compounding*, *concatenation*, or *composition*

The matrix product $T(d_{x1}, d_{y1}) \cdot T(d_{x2}, d_{y2})$ is :

$$\begin{bmatrix} 1 & 0 & d_{x1} \\ 0 & 1 & d_{y1} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & d_{x2} \\ 0 & 1 & d_{y2} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_{x1} + d_{x2} \\ 0 & 1 & d_{y1} + d_{y2} \\ 0 & 0 & 1 \end{bmatrix}$$

Matrix product is variously referred to as *compounding*, *concatenation*, or *composition*.

Matrix Question.

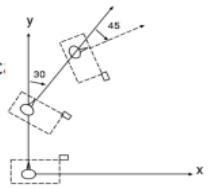
I sat in the car, and find the side mirror is 0.4m on my right and 0.3m in my front I started my car and drove 5m forward, turned 30 degrees to right, moved 5m forward again, and turned 45 degrees to the right, and stopped. What is the position of the side mirror now, relative to where I was sitting in the beginning?

Solution

TAN=O/A,SIN=O/H,CO=A/H

Solution

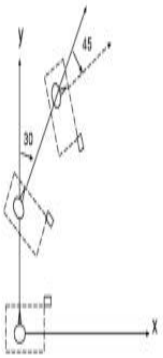
- The side mirror position is loc (0,4,0.3)
- The matrix of first driving forward 5m is



$$T_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix}$$

Solution

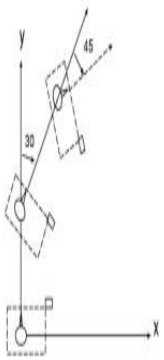
The matrix to turn to the right 30 and 45 degrees (rotating -30 and -45 degrees around the origin) are



$$R_1 = \begin{bmatrix} \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}, R_2 = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}, \text{respectively}$$

Solution

The local-to-global transformation matrix at the last configuration of the car is



$$TR_1TR_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 5 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The final position of the side mirror can be computed by TR_1TR_2 p which is around (2.89331, 9.0214)

Cos(60°) = 1/2 Sin(60°) = $\sqrt{3}/2$ Tan(60°)= $\sqrt{3}$	Cos(30°) = $\sqrt{3}/2$ Sin(30°) =1/2 Tan(30°)=1/$\sqrt{3}$	Cos(45°) = $\sqrt{2}/2$ Sin(45°) = $\sqrt{2}/2$ Tan(45°)=1

$$v'_x = a_1 v_x + b_1 v_y + c_1 v_z + d_1$$
$$v'_y = a_2 v_x + b_2 v_y + c_2 v_z + d_2$$
$$v'_z = a_3 v_x + b_3 v_y + c_3 v_z + d_3$$

$$\begin{bmatrix} v'_x \\ v'_y \\ v'_z \end{bmatrix} = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} + \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

$$\mathbf{v}' = \mathbf{M} \cdot \mathbf{v} + \mathbf{d}$$

$$x' = x.\cos\theta - y.\sin\theta$$
$$y' = x.\sin\theta + y.\cos\theta$$

Rewriting in matrix form gives us :

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Define the matrix $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$, $P' = R \cdot P$

3.5 A Simple Animation – Spinning

Animation is the generation of multiple frames with small differences between each.
Applying transformations over time

move 400 0 0 0 0
move 400 0 0 0 .25
move 400 0 0 0 .50
move 400 0 0 0 .75
move 400 0 0 0 1
move 400 0 0 0 m0 <-- *knob*

Lesson 3 Discussion Session

Implement Line Algorithm in C Program